

Summary Sorting Algorithms

Thursday, 28 October 2021 17:37

Bubble Sort:

Gehe durchs Array und vertausche wenn kleiner vorgänger.

$A = [a_1, a_2, a_3, \dots, a_n]$

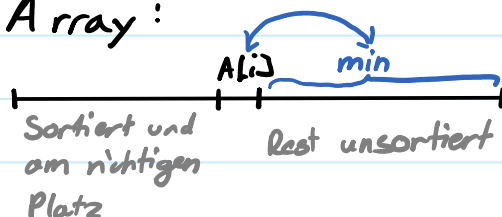
Laufzeit: $O(n^2)$

Speicher: $O(1)$

Selection Sort

Suche Minimum in restlichem Array.
tausche es mit iter stelle ein.

Array:



Invariante: $INV(i)$

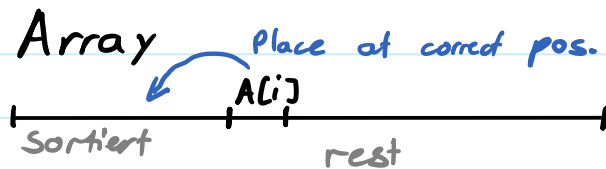
vor dem i ten Schritt ist das Array
von $A[1]$ bis $A[i-1]$ sortiert
und am richtigen Platz

Laufzeit: $O(n^2)$

Speicher: $O(1)$

Insertion Sort:

Platziere das i te Element an der richtigen Stelle in $A[1 \dots i-1]$



Invariante: $INV(i)$

Vor der i ten iteration ist das Array von $A[1]$ bis $A[i-1]$ sortiert.

Laufzeit: $O(n^2)$

Speicher: $O(1)$

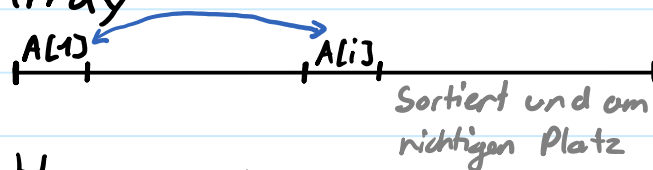
Heap Sort:

Variation zu Selection Sort:

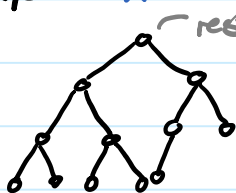
Selection Sort rückwärts und

Optimierung von Max zu Heap

Array:



Heap **max**:



Jeder Knoten ist grösser beide seiner Kinder.

Invariante: $INV(i)$

vor dem i ten Schritt ist das Array
von $A[1]$ bis $A[i-1]$ sortiert
und am richtigen Platz, zusätzlich
ist der Maximalwert des Heaps an der
Stelle $A[i]$

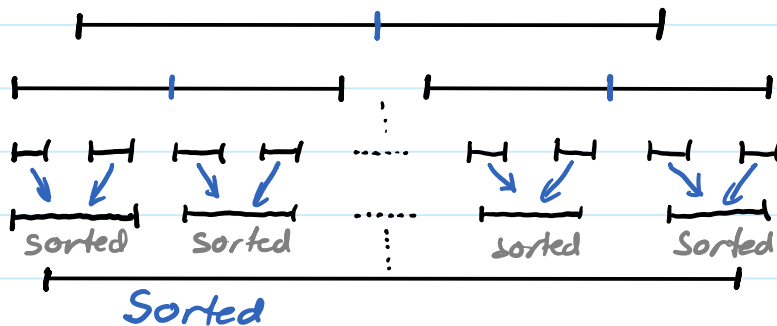
Laufzeit: $O(n \log n)$

Speicher: $O(1)$

Merge Sort

Teile das Array, bis es nur noch
Elemente mit Länge 1 gibt. Kombiniere
diese der Größe nach.

Array



Invariante:

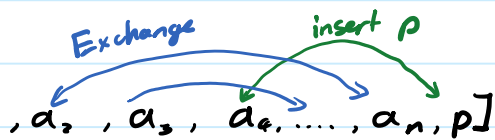
Vor merge von Array A und Array B
sind Array A und B sortiert.

Laufzeit: $O(n \log n)$

Speicher: $O(n)$

Quick Sort:

Finde Zahl p , tausche alle Elemente
sodass p an der richtigen Stelle liegt.
Wiederhole dies für die Teilliste kleiner p
und die grösser p

Array: 
The diagram shows an array $[a_1, a_2, a_3, a_4, \dots, a_n, p]$. A blue arrow labeled 'Exchange' points from a_4 to a_2 . A green arrow labeled 'insert p' points from p to a_4 .

Invariante:

p ist an der richtigen Stelle,
wenn das mit allen Elementen
passiert ist, ist das Array sortiert

Laufzeit: $O(n^2)$

Average: $O(n \log n)$

Speicher: $O(1)$